MATIÈRE	CLASSE	DURÉE	BARÈME	CHAPITRE(S)
Langage C	Licence 1 MIAGE	1 h 30	/20 — auto-évaluation	Pointeurs, tableaux, chaînes et structures

Consignes : Séance de travaux dirigés d'entraînement — exercices originaux rédigés par ResumeCI dans l'esprit des TD de Licence 1. Écris chaque programme en langage C, trace son exécution à la main, puis compare avec le corrigé.

Exercice 1 — Échange par pointeurs (4 pts)

a) Explication passage par valeur 1 pt · b) Fonction correcte 2 pts · c) Appel avec & 1 pt

On souhaite permuter le contenu de deux variables entières via une fonction.

a. Expliquer pourquoi la fonction `void echange(int a, int b)` ne peut pas permuter les variables de l'appelant.

b. Écrire la fonction `void echanger(int *a, int *b)` qui permute réellement les contenus.

c. Écrire le `main` qui teste la fonction avec 12 et 7.

✓ CORRIGÉ

a) En C, les paramètres sont passés **par valeur** : la fonction reçoit des copies locales. Modifier `a` et `b` dans la fonction ne change rien chez l'appelant.

b) En passant les **adresses**, la fonction écrit directement dans les variables :

```
void echanger(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

c)

```
#include <stdio.h>

int main(void) {
    int x = 12, y = 7;
    echanger(&x, &y);
    printf("x = %d, y = %d\n", x, y);  /* x = 7, y = 12 */
    return 0;
}
```

Exercice 2 — Tableau et arithmétique des pointeurs (4 pts)

a) Fonction par pointeurs 3 pts · b) 1 pt

Soit le tableau `int t[6] = {4, 9, 2, 11, 7, 5};`.

a. Écrire une fonction `int somme(int *t, int n)` qui retourne la somme des éléments **en utilisant l'arithmétique des pointeurs** (sans `t[i]`).

b. Que vaut l'expression `*(t + 3)` ? Justifier.

✓ CORRIGÉ

a) Le nom d'un tableau est l'adresse de son premier élément ; `*(t + i)` accède à `t[i]` :

```c
int somme(int *t, int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += *(t + i);        /* equivalent a t[i] */
    return s;
}
```

b) `*(t + 3)` est le 4e élément → 11 (les indices commencent à 0).

Exercice 3 — Chaînes de caractères (4 pts)

a) 2 pts · b) 2 pts

Sans utiliser `strlen` ni `strrev`, on manipule une chaîne via un pointeur.

a. Écrire `int longueur(char *s)` qui retourne le nombre de caractères avant `'\0'`.

b. Écrire `int compter(char *s, char c)` qui retourne le nombre d'occurrences de `c` dans `s`.

✓ CORRIGÉ

```c
int longueur(char *s) {
    int n = 0;
    while (*(s + n) != '\0')
        n++;
    return n;
}

int compter(char *s, char c) {
    int nb = 0;
    while (*s != '\0') {
        if (*s == c) nb++;
        s++;                /* avance le pointeur */
    }
    return nb;
}
```

Pour `compter("bonjour", 'o')` → 2. Le marqueur de fin `'\0'` est la seule borne fiable d'une chaîne C.

Exercice 4 — Structures (4 pts)

a) Déclaration + saisie 2,5 pts · b) Affichage 1,5 pt

On modélise un livre par la structure `struct Livre { char titre[50]; char auteur[40]; int annee; };`.

a. Déclarer un tableau de 3 livres et le remplir par saisie utilisateur.

b. Afficher ensuite la bibliothèque sous la forme « titre (auteur, année) ».

✓ CORRIGÉ

```c
#include <stdio.h>

struct Livre {
    char titre[50];
    char auteur[40];
    int annee;
};

int main(void) {
    struct Livre biblio[3];
    for (int i = 0; i < 3; i++) {
        printf("Titre : ");
        scanf(" %49[^\n]", biblio[i].titre);
        printf("Auteur : ");
        scanf(" %39[^\n]", biblio[i].auteur);
        printf("Annee : ");
        scanf("%d", &biblio[i].annee);
    }
    for (int i = 0; i < 3; i++)
        printf("%s (%s, %d)\n",
               biblio[i].titre, biblio[i].auteur, biblio[i].annee);
    return 0;
}
```

L'accès aux champs se fait par `biblio[i].titre` ; `%49[^\n]` lit une ligne entière avec espaces en protégeant le tableau.

Exercice 5 — Extremums et positions (4 pts)

Saisie 1 pt · Recherche min/max 2 pts · Positions 1 pt

Écrire un programme qui lit 8 entiers dans un tableau puis affiche la plus petite valeur, la plus grande valeur et leurs positions (indices) respectives.

En cas d'égalité, on retient la première position rencontrée.

✓ CORRIGÉ

```c
#include <stdio.h>

int main(void) {
    int t[8], imin = 0, imax = 0;
    for (int i = 0; i < 8; i++)
        scanf("%d", &t[i]);
    for (int i = 1; i < 8; i++) {
        if (t[i] < t[imin]) imin = i;
        if (t[i] > t[imax]) imax = i;
    }
    printf("Min : %d (position %d)\n", t[imin], imin);
    printf("Max : %d (position %d)\n", t[imax], imax);
    return 0;
}
```

On mémorise les **indices** (`imin`, `imax`) plutôt que les valeurs : un seul parcours suffit et l'égalité conserve naturellement la première position (comparaison stricte).